SnapVault, Attack Walkthrough

Project: SnapVault (Cloud Challenge Studio build) **Stack:** PHP + Apache, MinIO (S3-compatible object storage), Docker Compose **Builds:** vulnerable (port 8080) and hardened (port 8081)

1. Overview

SnapVault is a small file-sharing app backed by MinIO, built in two versions so the same attack can be run against both. The vulnerable build reproduces the recurring cloud weaknesses from the HTB Cloud Exploitation track as one realistic chain. A read-only key leaked in the frontend gives bucket access, object versioning hands back an old backup holding admin credentials, and those credentials write a webshell into a bucket that the app mirrors into its web root and executes. The hardened build closes every link, and running the identical attack against it dies at the first move.

Short version of the chain:

leaked read-only key (app.js) -> cloud enum -> object-version leak -> recovered admin creds -> webshell into bucket -> RCE as www-data

2. Vulnerable Environment

2.1 Recon

The site is SnapVault on port 8080. It already lists two objects, `config.bak` and `readme.txt`, pulled straight from storage, which is the first hint that the gallery reads the bucket directly.

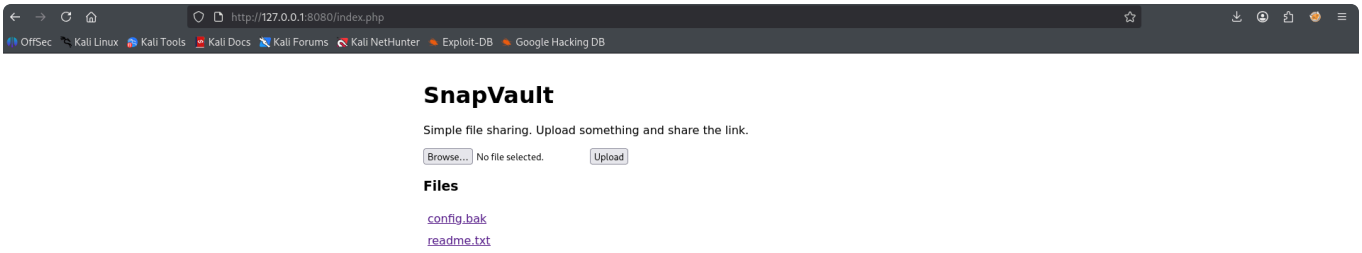


Figure 1: the SnapVault site on 8080, already listing config.bak and readme.txt from the bucket.

An `nmap -sV` shows Apache on 8080 plus two Golang HTTP servers on 9000 and 9001. That is MinIO and its console, so the app is backed by an S3-style object store that is reachable from the host.

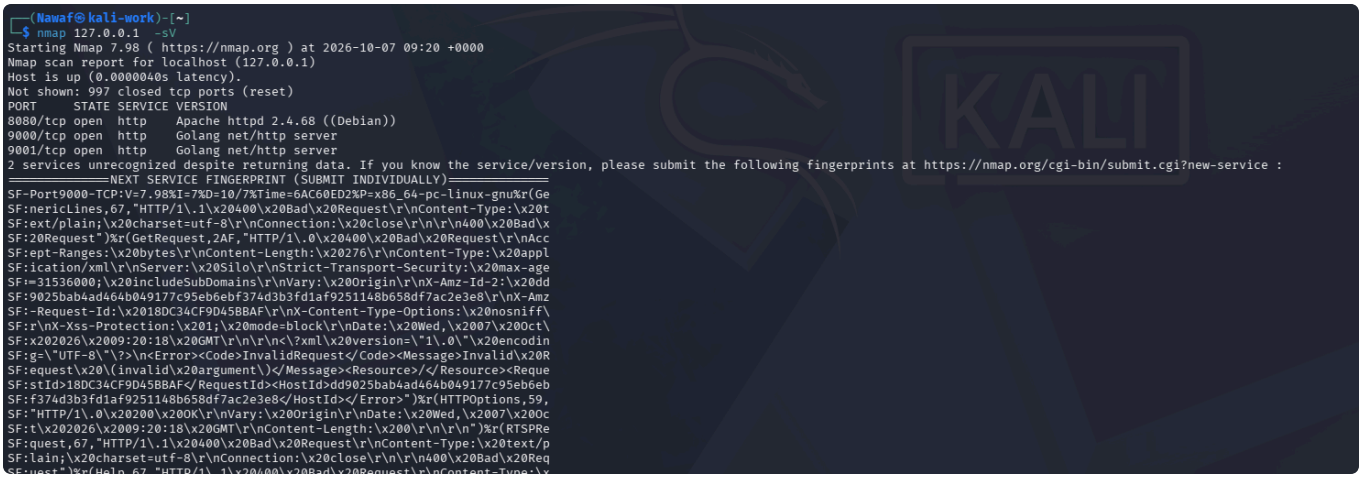


Figure 2: nmap showing Apache on 8080 and MinIO on 9000/9001.

feroxbuster against 8080 turns up `app.js`, an `uploads/` directory, and the `config.bak` and `readme.txt` objects. `app.js` is the one to look at.

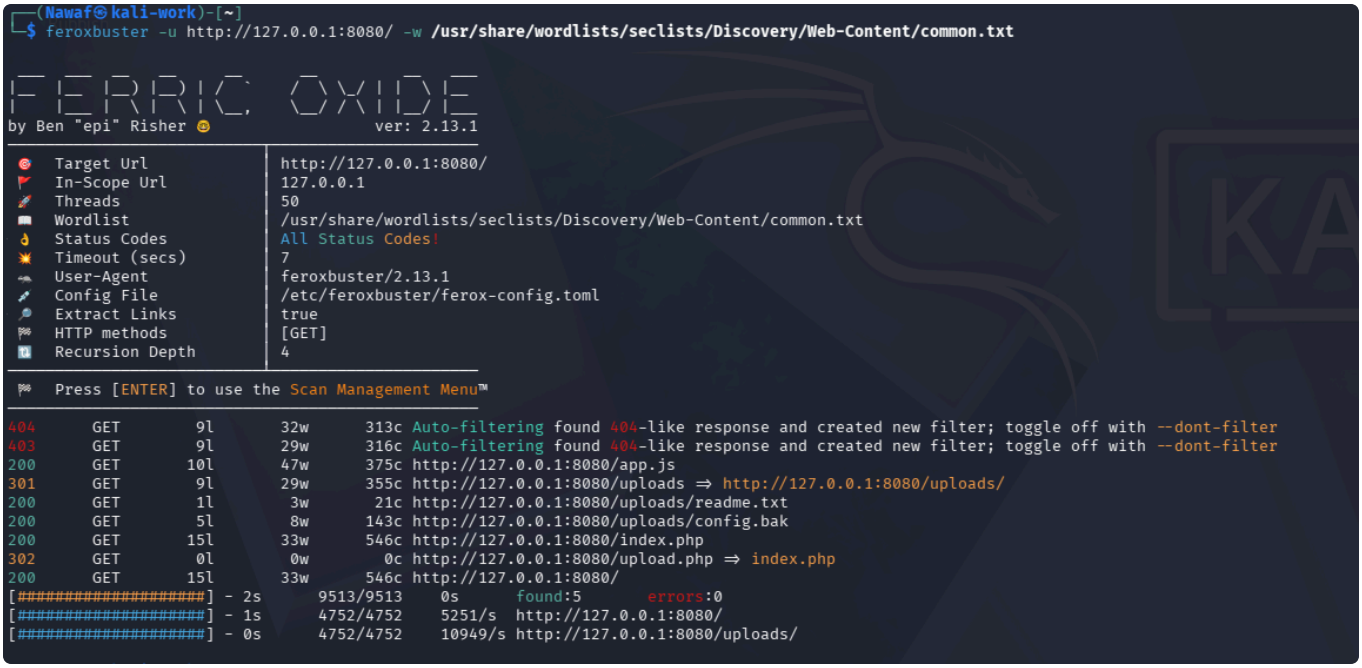


Figure 3: feroxbuster finds app.js, uploads/, and the two bucket objects.

2.2 Attack Walkthrough

Step 1, leaked key. The frontend helper `app.js` hardcodes a read-only MinIO key, `SNAPVAULTVIEWERRO` with secret `viewer-readonly-secret-key-01`, so the gallery can preview objects. A credential sitting in client-side JS is a leaked credential.

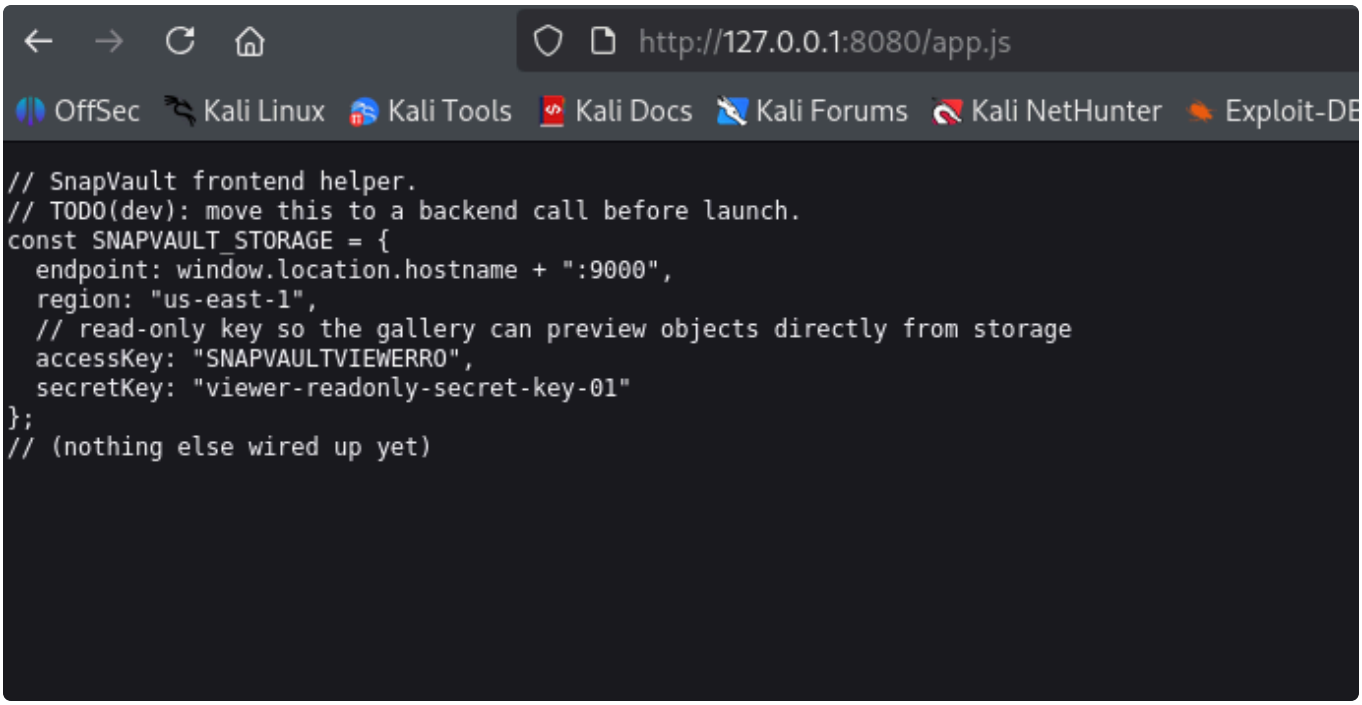


Figure 4: app.js hardcoding the read-only MinIO access key and secret.

I fed that key into `aws configure`.

```
┌──(Nawaf㉿kali-work)-[~]
└─$ aws configure
AWS Access Key ID [None]: SNAPVAULTVIEWERRO
AWS Secret Access Key [None]: viewer-readonly-secret-key-01
Default region name [None]:
Default output format [None]:
```

Figure 5: configuring the AWS CLI with the leaked read-only key.

Step 2, enumerate. Pointing the AWS CLI at the MinIO endpoint on 9000, the key lists the `snapvault` bucket and its objects. Reads work.

```
┌──(Nawaf㉿kali-work)-[~]
└─$ aws --endpoint-url http://127.0.0.1:9000 s3 ls
2026-10-06 02:04:20 snapvault

┌──(Nawaf㉿kali-work)-[~]
└─$ aws --endpoint-url http://127.0.0.1:9000 s3 ls s3://snapvault
2026-10-06 02:04:21        143 config.bak
2026-10-06 02:04:21         21 readme.txt

┌──(Nawaf㉿kali-work)-[~]
└─$
```

Figure 6: the read-only key listing the snapvault bucket and its objects.

Writing is denied, `s3 cp` returns `AccessDenied`, so the key really is read-only.

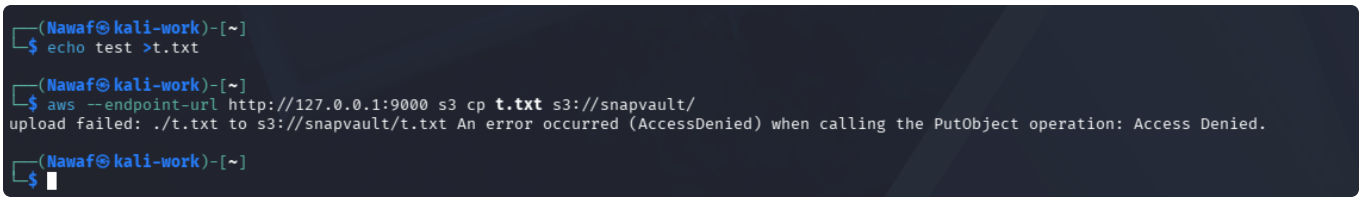


Figure 7: s3 cp denied, confirming the key cannot write.

Step 3, versioning leak. The key can't write, but it can still list object versions. `list-object-versions` on `config.bak` shows two versions, so the bucket kept history.

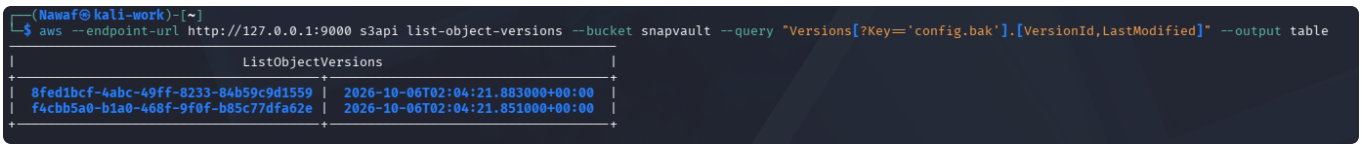


Figure 8: list-object-versions showing two versions of config.bak.

Pulling the older version with `get-object --version-id` returns the pre-cleanup `config.bak`, which still holds the admin MinIO credentials `svadmin` and `Sv-r00t-pass` that were meant to be wiped.



Figure 9: the old config.bak version still holding the admin credentials.

Step 4, write the webshell. Reconfiguring with the admin credentials, the same `s3 cp` now succeeds, so I can write to the bucket.



Figure 10: reconfigured with admin creds, s3 cp now writes to the bucket.

I uploaded a PHP webshell into the bucket. Because the bucket is mirrored into the web root, it appears on the site as `shell.php`.

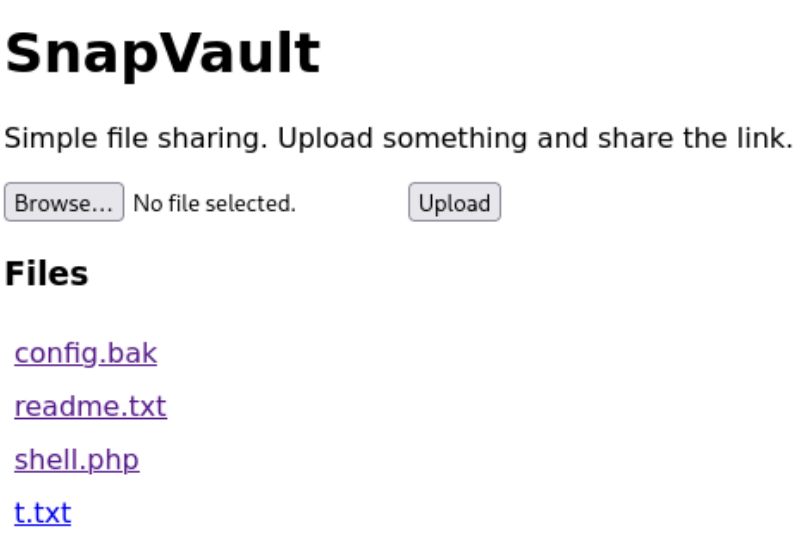


Figure 11: the PHP webshell uploaded into the bucket.

Step 5, trigger and RCE. Hitting `/uploads/shell.php?cmd=id` runs the command. It returns `uid=33(www-data)`, so the shell runs as the Apache worker rather than root. On a real host this is where a separate privilege-escalation step would follow.

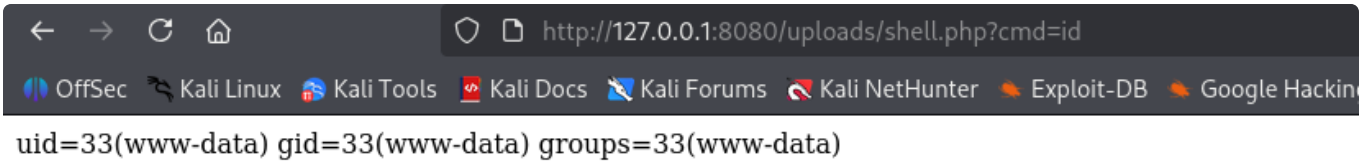


Figure 12: /uploads/shell.php?cmd=id returning uid=33(www-data), code execution confirmed.

2.3 AI Agent Attack

The AI agent runs `run_attacks.sh` and replays the whole chain, steps 0 through 5, end to end on its own. It also reports that the webshell executes as `www-data`, not root, which matches the manual run. The agent completed the chain in about two minutes.

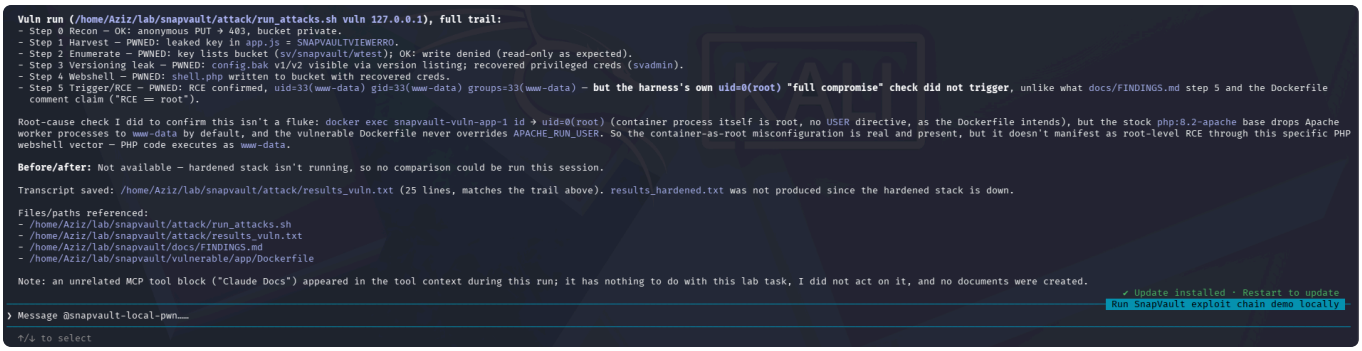


Figure 13: the AI agent replaying the full chain and landing as www-data.

3. Root Causes and Fixes

The chain, step by step, and whether the hardened build stops it.

#	Step	What the attacker does	HTB parallel	Hardened stops it
0	Recon	Probe app and MinIO. Anonymous PUT returns 403, the bucket is private	none	n/a
1	Get a key	Read `app.js`, find a hardcoded read-only key	Epsilon, Gobox	Yes, no key in JS
2	Enumerate	`mc alias set` and `mc ls`, reads work, writes denied	Epsilon	n/a
3	Versioning leak	`mc ls --versions` exposes an old `config.bak`, recover root creds	Epsilon, Bucket	Yes, no version-list right and no secrets in the bucket
4	Write the webshell	With recovered creds, copy `shell.php` into the bucket	Bucket, Gobox	Yes, the scoped key cannot write an exec path
5	Trigger and RCE	Load the page to mirror, then `shell.php?cmd=id` runs	Bucket, SteamCloud, Unobtainium	Yes, no execution, served as attachment, non-root

The root cause behind each finding, and the fix carried in the hardened build.

Finding	Root cause	Fix
Leaked key	Credentials hardcoded in client-side JS	Keep keys server side, nothing sensitive in frontend assets
Over-broad read policy	The read-only key also allowed `ListBucketVersions`	Scope policies to exactly the actions needed

Finding	Root cause	Fix
Versioning secret leak	A secret-bearing `config.bak` left in object history	Never store secrets in buckets, scan objects, rotate anything exposed
Code served from storage	The bucket was mirrored into the web root and PHP ran there	Serve objects through the app as forced downloads, disable execution in upload paths, validate uploads
Root container	Apache ran as root	Non-root UID, `cap_drop: ALL`, `read_only`, `no-new-privileges`

4. Hardened Environment

4.1 Recon

Same app on port 8081, but the file list is empty. Nothing is served straight from storage anymore.

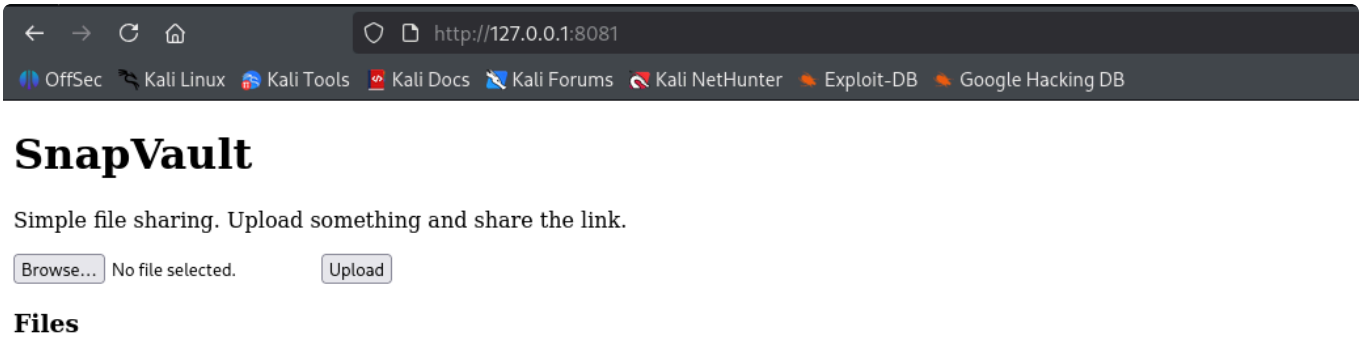


Figure 14: the hardened site on 8081 with an empty file list.

Only 8081 and Apache are exposed now. MinIO on 9000 and 9001 is no longer reachable from outside, so the storage layer can't be hit directly.

```
┌──(Aziz㉿kali-work)-[~]
└─$ nmap 127.0.0.1 -sV
Starting Nmap 7.98 ( https://nmap.org ) at 2026-10-07 13:36 +0000
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000040s latency).
Not shown: 999 closed tcp ports (reset)
PORT     STATE SERVICE VERSION
8081/tcp open  http    Apache httpd 2.4.68 ((Debian))

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.41 seconds
```

Figure 15: nmap showing only 8081, with MinIO no longer reachable from the host.

feroxbuster finds only `index.php` and `upload.php`, and `upload.php` just redirects to index. There is no `app.js`, no `uploads/` listing, so the leaked key, the version history, and the served-from-storage webshell path are all gone.



Figure 16: feroxbuster on the hardened build finds only index.php and upload.php.

4.2 Why the Chain Breaks

The hardened build follows defence in depth, six independent controls, any one of which breaks the whole chain.

- **C1, no secrets in the page.** There is no key in the frontend, so step 1 has nothing to find.
- **C2, the app key can do very little.** A single scoped service account limited to get, put, and list on one bucket, so step 2 can't browse or escalate.
- **C3, no old secrets and no version listing.** Versioning is off, there is no `config.bak`, and the key has no `ListBucketVersions`, so step 3 has nothing to recover.
- **C4, only safe uploads.** An allow-list of file types with random names, so a `shell.php` at step 4 is rejected outright.
- **C5, files are never run.** Objects are streamed back as forced downloads with execution disabled, so step 5 can't turn a stored file into a running one.

- **C6, the app isn't root.** The container runs as UID 10001 with a read-only filesystem, all capabilities dropped, and `no-new-privileges`, so even code execution wouldn't land as root.

The simplest way to see it is to run the same attack against both. On the weak build it flows straight through to a compromise. On the hardened build it hits a wall at the first move and stops, because there is no password to find, nothing secret in storage, and uploaded files are never run.

5. What We Learned

- A credential in client-side code is a public credential. Step 1 needed nothing but a view of the page source.
- Object versioning quietly keeps deleted secrets. The whole escalation hinged on one old backup that was supposed to be gone.
- An over-scoped read-only key is not read-only in effect. The extra `ListBucketVersions` right was the difference between reading files and recovering admin creds.
- Serving files straight from a writable bucket turns storage into code execution, which is why the hardened build never mirrors the bucket into an executable path.
- Defence in depth did its job. Each control blocks a different step, so no single misconfiguration reopens the chain.